



Course: Optimization Technique

Lecturer: Dr. Ahmed Magdy

AIEP - Third year

REPORT:

Diabetes prediction

Prepared by:

Abdelrahman gamal hussien ali



1. Executive Summary

This project builds a machine learning model to predict diabetes status from patient-related features such as age, BMI, hypertension, heart disease, HbA1c level, blood glucose level, gender, and smoking history. The main challenge in this dataset is the class imbalance: most records are non-diabetic, while diabetic cases represent a much smaller part of the data. Because of that, the model should not only achieve high accuracy; it also needs to detect diabetic cases well without producing too many false alarms.

The work started with a base XGBoost model, then improved it using a Balanced Gray Wolf Optimizer. The optimizer was designed to search for stronger XGBoost hyperparameters while keeping the model balanced. Instead of pushing recall alone, the fitness function combined F1-score, F2-score, recall, precision, accuracy, PR-AUC, and ROC-AUC. This made the optimized model more stable and more suitable for a practical diabetes prediction task.

After optimization, the model improved test accuracy from 0.9017 to 0.9277 and improved F1-score from 0.6225 to 0.6815. Precision also improved from 0.4706 to 0.5573, which means the optimized model reduced many false positive diabetes predictions. Recall decreased from 0.9192 to 0.8768, but the final result is more balanced and easier to trust because the model is no longer over-predicting the positive class.

2. Dataset Description

The dataset contains clinical and demographic information for diabetes prediction. The original file contained 100,000 records. After checking for duplicates, 3,854 duplicated rows were removed, leaving 96,146 records for modeling.

Feature	Description
gender	Patient gender category
age	Patient age
hypertension	Whether the patient has hypertension
heart_disease	Whether the patient has heart disease
smoking_history	Smoking category reported in the dataset
bmi	Body Mass Index
HbA1c_level	HbA1c blood test level
blood_glucose_level	Blood glucose level
diabetes	Target variable: 0 for non-diabetic and 1 for diabetic

The target distribution is imbalanced. Before duplicate removal, the dataset had 91,500 non-diabetic records and 8,500 diabetic records, which means diabetic cases represented about 8.50% of the full dataset. After splitting the cleaned dataset, the training set contained 56,103 non-diabetic cases and 5,429 diabetic cases, giving a class-weight ratio of about 10.33.

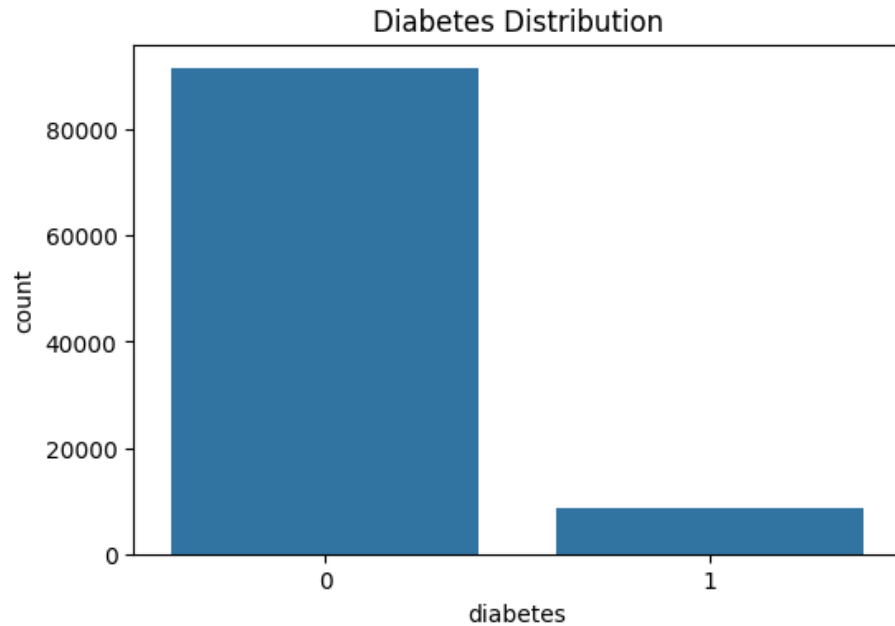


Figure 1. Distribution of the diabetes target before model training.

3. Data Preparation and Modeling Workflow

The preprocessing stage was kept straightforward and consistent. Duplicate rows were removed, the index was reset, and the data was split into training, validation, and test sets using stratification so that the class distribution stayed similar across the splits.

Split	Number of records
Training set	61,532
Validation set	15,384
Test set	19,230

Several engineered features were added to give the model more useful relationships between variables. These features were not created to replace the original variables, but to make the important medical signals easier for the model to learn.

Engineered feature	Meaning
age_bmi	Age multiplied by BMI
age_hb1c	Age multiplied by HbA1c level
bmi_glucose	BMI multiplied by blood glucose level
hb1c_glucose	HbA1c level multiplied by blood glucose level
risk_factor_count	Hypertension plus heart disease
glucose_hb1c_ratio	Blood glucose level divided by HbA1c level

Numerical columns were handled using median imputation followed by standard scaling. Categorical columns were handled using most-frequent imputation and one-hot encoding. Binary columns were imputed using the most frequent value and kept as binary indicators. The same preprocessing pipeline was used before and after optimization, so the comparison stayed fair.

Course: Optimization Technique

Lecturer: Dr. Ahmed Magdy

AIEP - Third year

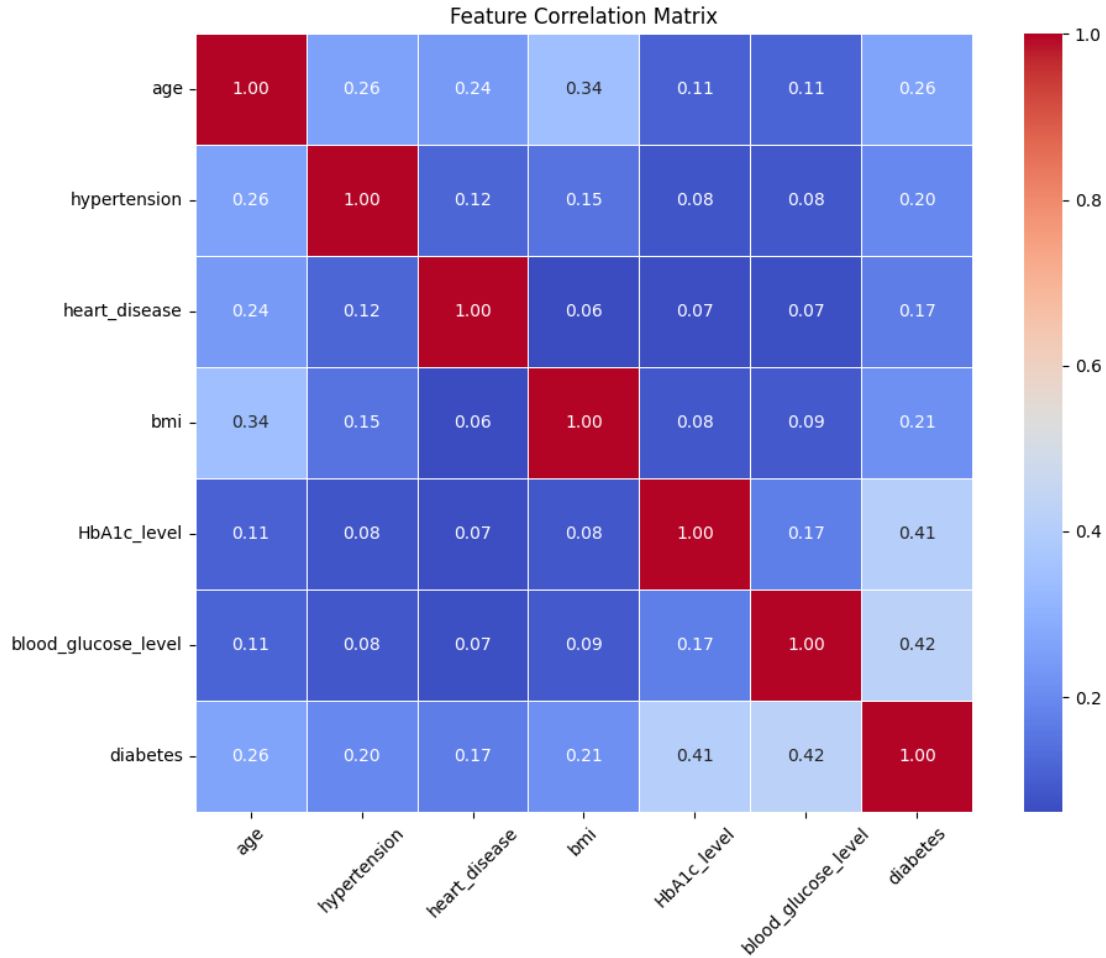


Figure 2. Correlation matrix for the main numerical variables.

4. Model Before Optimization: Base XGBoost

The base XGBoost model was trained with fixed hyperparameters and the original imbalance ratio using $\text{scale_pos_weight} = 10.33$. This helped the model pay more attention to diabetic cases, but it also made the model aggressive toward the positive class.

Metric	Base XGBoost Result
Training Accuracy	0.9103
Test Accuracy	0.9017
Precision	0.4706
Recall	0.9192
F1-score	0.6225
F2-score	0.7720
ROC-AUC	0.9776
PR-AUC	0.8843

The base model achieved high recall, meaning it detected most diabetic cases. However, precision was low, which means many records predicted as diabetic were actually non-diabetic. This was the main reason for applying optimization: the model needed to stay sensitive to diabetes cases but become more controlled in its positive predictions.

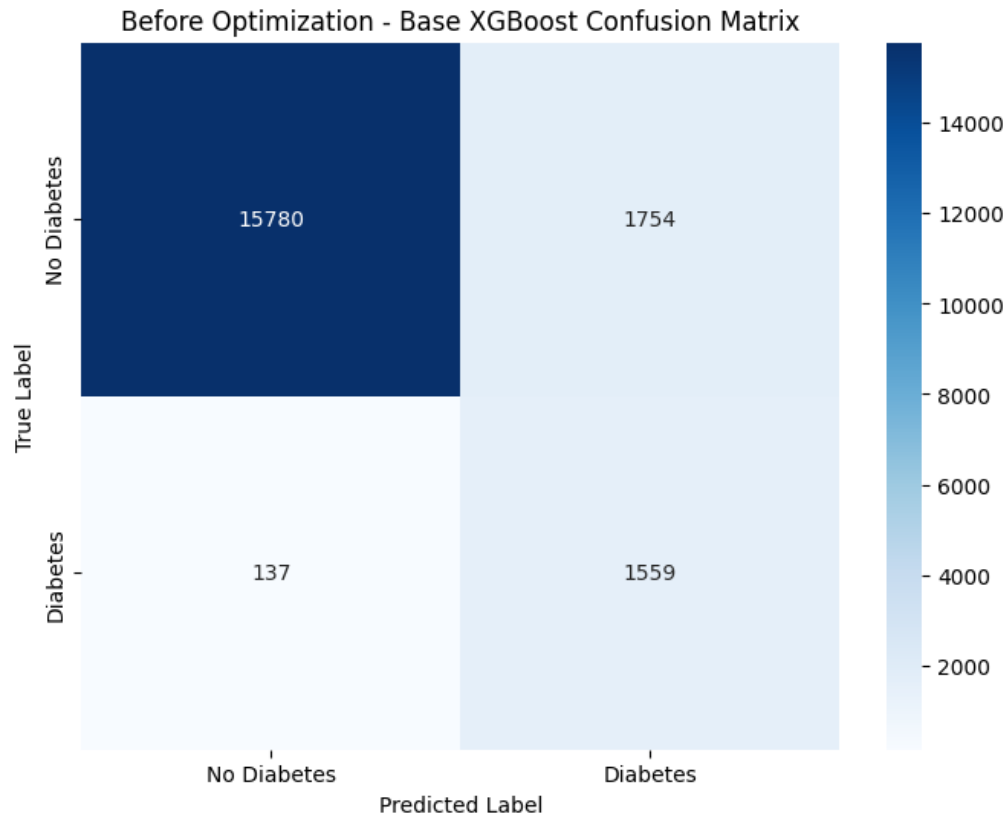


Figure 3. Confusion matrix for the base XGBoost model before optimization.

5. Balanced Gray Wolf Optimization for XGBoost

The optimization part is the main improvement in this project. Gray Wolf Optimizer was used to tune XGBoost hyperparameters automatically. The idea is simple: each wolf represents one possible XGBoost configuration. During the search, the best three wolves are named alpha, beta, and delta. The rest of the population updates its position by moving around these three leading solutions. In this project, a position is not a point in a physical space; it is a set of XGBoost hyperparameter values.

5.1 Optimized Hyperparameter Vector

Each wolf position was represented as the following vector:

$X = [n_estimators, max_depth, learning_rate, subsample, colsample_bytree, min_child_weight, gamma, reg_alpha, reg_lambda, max_delta_step, scale_pos_weight_multiplier]$

The last value, `scale_pos_weight_multiplier`, was especially important. The original imbalance ratio was 10.33. If this value is used too strongly, XGBoost becomes too sensitive to the diabetic class and produces many false positives. The optimizer was allowed to adjust this weight through a multiplier, and the best final `scale_pos_weight` became 6.6297. This shows that the optimized model did not ignore imbalance, but it reduced the strength of the class weighting to make predictions more balanced.

Hyperparameter	Search range
n_estimators	350 to 900
max_depth	2 to 5
learning_rate	0.020 to 0.100
subsample	0.80 to 1.00
colsample_bytree	0.80 to 1.00
min_child_weight	2 to 9
gamma	0.0 to 2.5
reg_alpha	0.0 to 3.0
reg_lambda	2.0 to 12.0
max_delta_step	0.0 to 4.0
scale_pos_weight_multiplier	0.55 to 1.35

5.2 GWO Position Update Equations

The optimizer updates each wolf using the alpha, beta, and delta wolves. The control parameter a decreases over the iterations, so the search starts with more exploration and gradually becomes more focused around the best regions.

$$a = 2 - (2t / T)$$

$$A = 2ar_1 - a, \quad C = 2r_2$$

$$D_{\alpha} = |C_1 * X_{\alpha} - X|, \quad X_1 = X_{\alpha} - A_1 * D_{\alpha}$$

$$D_{\beta} = |C_2 * X_{\beta} - X|, \quad X_2 = X_{\beta} - A_2 * D_{\beta}$$

$$D_{\delta} = |C_3 * X_{\delta} - X|, \quad X_3 = X_{\delta} - A_3 * D_{\delta}$$

$$X(t+1) = (X_1 + X_2 + X_3) / 3$$

Here, X is the current wolf position, X_{α} is the best solution found so far, X_{β} is the second best solution, and X_{δ} is the third best solution. r_1 and r_2 are random values between 0 and 1. This update rule allows the optimizer to explore different hyperparameter combinations first, then gradually concentrate around the best-performing configurations.

5.3 Balanced Fitness Function

The fitness function was not written to maximize recall only. A recall-only objective would probably increase the number of detected diabetic cases, but it would also create too many false positives. For this project, the objective was balanced: high recall was still important, but the optimizer also had to protect precision, F1-score, accuracy, and ranking quality.

$$F2 = (5 * Precision * Recall) / (4 * Precision + Recall)$$

$$Score(X) = 0.28F2 + 0.24F1 + 0.20Recall + 0.12Precision + 0.08PR-AUC + 0.05Accuracy + 0.03ROC-AUC - Penalties$$

$$Penalties = 0.50\max(0, 0.50 - Precision) + 0.45\max(0, 0.66 - F1) + 0.35\max(0, 0.91 - Accuracy) + 0.30\max(0, 0.90 - Recall) + 0.20\max(0, (Recall - Precision) - 0.43)$$

$$Fitness(X) = -Score(X)$$

The optimizer minimizes $Fitness(X)$, so the score is multiplied by -1. The penalty terms are important because they stop the optimizer from choosing a model that looks good in only one metric. For example, if precision drops below 0.50 or if the gap between recall and precision becomes too large, the configuration is penalized. This is why the final model is more balanced than the base model.

5.4 Optimization Procedure

Setting	Value used in notebook
Population size	14 wolves
Main GWO iterations	18
Local search trials	20
Random seed	42
Prediction method	Normal model.predict(), no threshold tuning
Validation role	Used only for optimizer fitness evaluation

After the main GWO loop, a small local search was applied around the best solution. This final step made small changes near the best wolf position and kept the change only if it improved the balanced validation score. This helped refine the final hyperparameters without changing the general optimization method.

Hyperparameter	Best value found by GWO
n_estimators	685
max_depth	4
learning_rate	0.0203356
subsample	0.9980943
colsample_bytree	0.8000
min_child_weight	2.9787449
gamma	1.8668879
reg_alpha	1.0975298
reg_lambda	7.7380319
max_delta_step	1.2459115
scale_pos_weight	6.629716

6. Model After Optimization: Balanced GWO-XGBoost

The final optimized model was retrained using the combined training and validation data, then evaluated once on the untouched test set. This gives a fair final result because the test set was not used during the GWO search.

Metric	Balanced GWO-XGBoost Result
Training Accuracy	0.9325
Test Accuracy	0.9277
Precision	0.5573
Recall	0.8768
F1-score	0.6815
F2-score	0.7866
ROC-AUC	0.9778
PR-AUC	0.8849

The optimized model achieved a better balance than the base XGBoost model. The most important improvement is the increase in precision and F1-score. The model still detects most diabetic cases, but it is less likely to label non-diabetic records as diabetic. The training-test accuracy difference was only 0.0048, so there was no strong sign of overfitting in the final result.

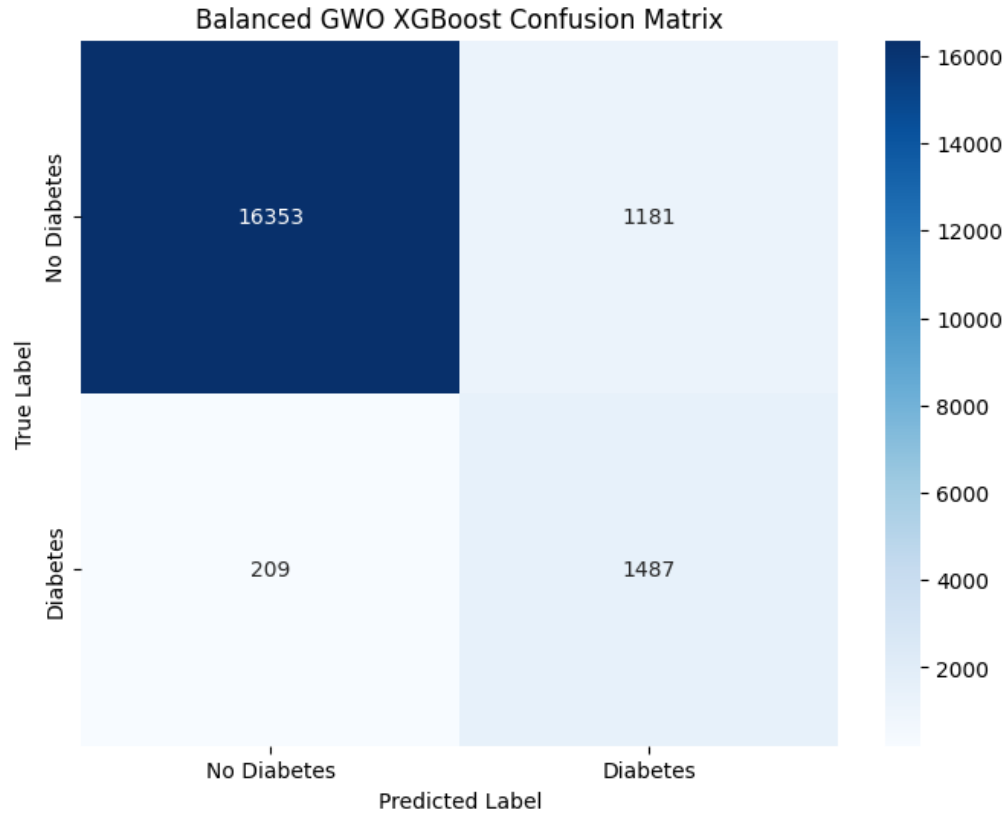


Figure 4. Confusion matrix for the Balanced GWO-XGBoost model.

7. Before and After Optimization Comparison

Metric	Before XGBoost	After GWO-XGBoost	Change
Training Accuracy	0.9103	0.9325	+0.0222
Test Accuracy	0.9017	0.9277	+0.0260
Precision	0.4706	0.5573	+0.0867
Recall	0.9192	0.8768	-0.0424
F1-score	0.6225	0.6815	+0.0590
F2-score	0.7720	0.7866	+0.0146
ROC-AUC	0.9776	0.9778	+0.0002
PR-AUC	0.8843	0.8849	+0.0006

The comparison shows the real effect of the optimizer. The base model had very high recall, but this came with weak precision. After GWO optimization, recall dropped slightly, but precision, F1-score, F2-score, and accuracy all improved. In other words, the optimized model became less extreme and more useful as a balanced diabetes prediction model.

Course: Optimization Technique

Lecturer: Dr. Ahmed Magdy

AIEP - Third year

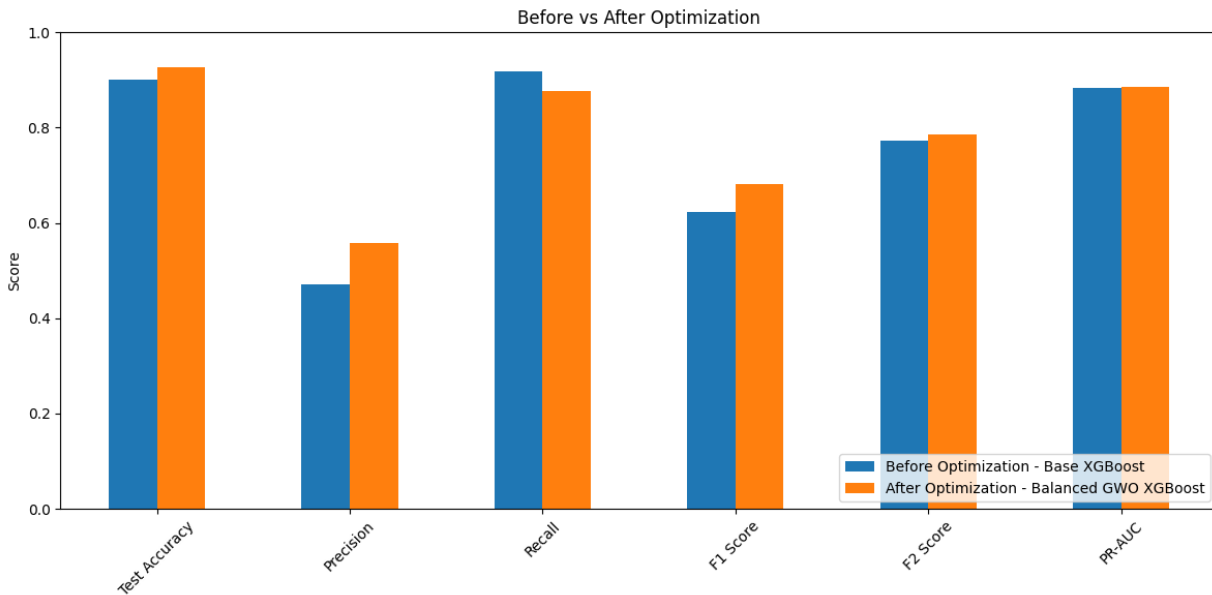


Figure 5. Before and after optimization comparison.

8. Model Interpretation

XGBoost also provides feature importance values, which help explain which inputs contributed most to the model decisions. The strongest predictors are expected to be related to blood glucose, HbA1c, and their engineered interactions, because these variables are directly connected with diabetes risk in the dataset. The engineered features helped the model capture combined effects rather than relying only on the original columns separately.

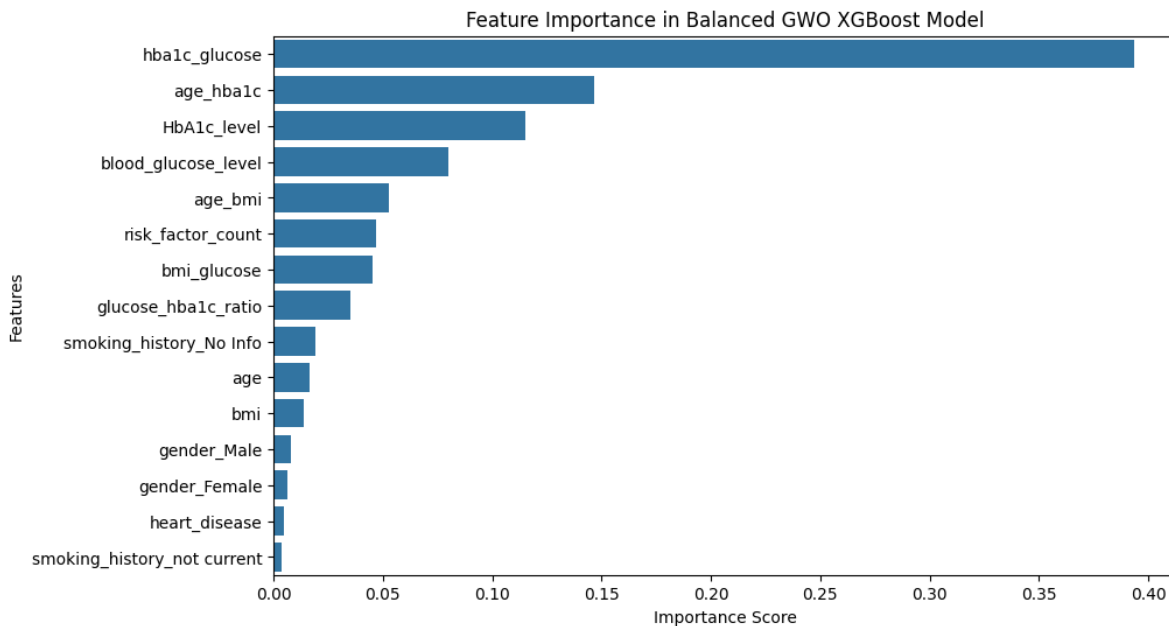


Figure 6. Top feature importance values from the optimized XGBoost model.

The ROC curve also confirms that the model separates diabetic and non-diabetic records very well at the probability-ranking level. The final ROC-AUC was 0.9778, which means the model gives useful probability ordering even though the final class prediction still depends on the default XGBoost decision rule.

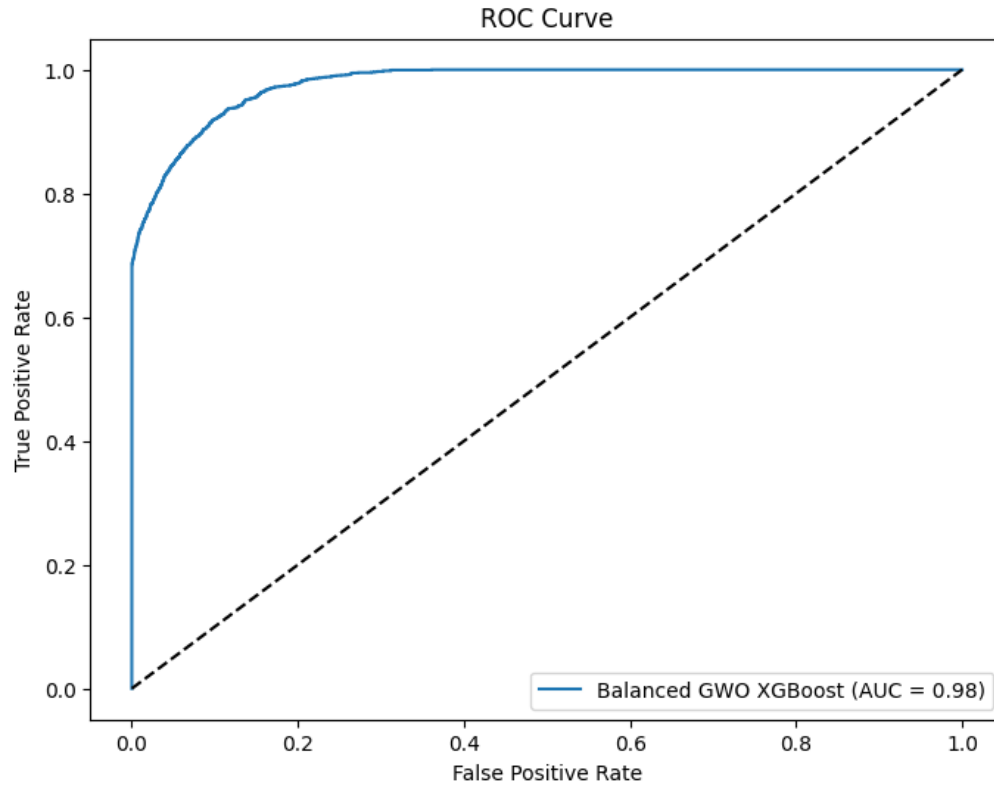


Figure 7. ROC curve for the final Balanced GWO-XGBoost model.

9. Conclusion

The Balanced GWO-XGBoost model is a clear improvement over the base XGBoost model for this project. The optimizer did not simply chase the highest recall. Instead, it corrected the model behavior by reducing excessive positive predictions and improving the overall balance between precision, recall, F1-score, and accuracy.

The final model reached 0.9277 test accuracy, 0.6815 F1-score, 0.7866 F2-score, 0.9778 ROC-AUC, and 0.8849 PR-AUC. These results show that the model is strong at separating the two classes and more balanced after optimization. For a diabetes prediction task, this balance is important because missing diabetic cases is harmful, but too many false alarms also reduce the usefulness of the system.

A Random Forest model was also trained as a general benchmark in the notebook, but the main experiment in this report focuses on the effect of applying Gray Wolf Optimization to XGBoost. The before-and-after comparison shows that GWO helped XGBoost become more reliable and more controlled without using threshold tuning.

10. Future Improvements

- Test the optimized model on an external dataset to check generalization.
- Use cross-validation inside the optimizer if more computation time is available.



Course: Optimization Technique

Lecturer: Dr. Ahmed Magdy

AIEP - Third year

- Compare Balanced GWO against PSO, GSA, and hybrid PSO-GSA using the same fitness function.
- Add explainability tools such as SHAP to explain individual predictions.